SISTEMAS DISTRIBUÍDOS 1 — TRABALHO PRÁTICO 1

Avaliação de Desempenho de uma Aplicação CRUD sob Carga Concorrente

Análise experimental com Python (Flask), PostgreSQL e Apache JMeter
Cenários de Leitura e Escrita, Redução de Dataset (100k vs. 20k) e Impacto de Índices

---

Aluno: _____ | Outubro de 2026

1. Objetivo e Escopo do Trabalho

O que foi proposto no enunciado da disciplina

Parte 1 — Projeto e Implementação

- Implementar uma aplicação **CRUD completa** envolvendo Clientes, Servidor Web e Banco de Dados.
- Popular o banco com um dataset de pelo menos **50.000 registros** (utilizamos **100.000 registros**).
- Projetar a arquitetura da solução em camadas.
- Simular múltiplos clientes simultâneos utilizando o **Apache JMeter**.

Parte 2 — Experimentos e Questões

- Avaliar o desempenho nos **3 cenários de carga**:
  - **Cenário A:** 50% Leituras / 50% Escritas
  - **Cenário B:** 75% Leituras / 25% Escritas
  - **Cenário C:** 25% Leituras / 75% Escritas
- Investigar e responder **3 questões práticas**:
  1. O que ocorre ao reduzir o dataset para **20k registros**?
  2. Qual o efeito de adicionar **índices no BD**? Qual escolher?
  3. Como a escolha entre **SQL e NoSQL** impacta o sistema?

2. A Aplicação Desenvolvida (CRUD)

Sistema Web e API REST de cadastro de pessoas em Python Flask + PostgreSQL

Modelo de Dados (`public.pessoas`)

- `id` — Chave primária (`INTEGER PRIMARY KEY`)
- `nome` e `email` — Campos obrigatórios
- `telefone` e `cidade` — Dados de contato/localização
- `data_cadastro` (`DATE`) e `status` (*ativo, inativo, pendente, bloqueado*)

Geração de Dados (`pessoas.py`)

- Script Python gerador de registros sintéticos realistas.
- **Base Principal:** 100.000 registros importados via `\copy` .
- **Base Reduzida:** 20.000 registros para comparação.

Endpoints Utilizados nos Testes

- **Operação de Leitura:** `GET /api/pessoas`
  - Retorna os 100 registros mais recentes em JSON (`ORDER BY id DESC LIMIT 100`).
  - Aceita busca textual opcional via `?q=termo` (com `ILIKE '%termo%'`).
- **Operação de Escrita:** `POST /api/pessoas`
  - Recebe JSON com os dados de uma nova pessoa.
  - Executa `INSERT INTO public.pessoas ... RETURNING id` com commit imediato (HTTP 201).
- A aplicação também conta com interface HTML e rotas de edição (`UPDATE`) e exclusão (`DELETE`).

3. Arquitetura da Solução

Organização cliente-servidor em 3 camadas e fluxo de comunicação

CAMADA 1

Clientes (Carga)

Apache JMeter 5.6.3 / Browser

- **1.200 clientes simultâneos** disparando requisições concorrentes.
- **Ramp-up:** 10 segundos.
- Simula os 3 perfis de tráfego: **50/50**, **75/25** e **25/75** (Leitura/Escrita).

HTTP / REST

⇄

JSON
Porta 5000

CAMADA 2

Servidor Web / API

Python 3.14 + Flask 3.1.3

- **Leitura:** `GET /api/pessoas`
- **Escrita:** `POST /api/pessoas`
- **Comunicação Síncrona:** abre e fecha 1 conexão por requisição via `psycopg2`.

TCP / SQL

⇄

psycopg2
Porta 5432

CAMADA 3

Banco de Dados

PostgreSQL 16

- Tabela relacional `public.pessoas`.
- **Datasets testados:** 100.000 e 20.000 registros.
- **Índices B-Tree:** `cidade`, `data_cadastro` e `status`.

Fluxo da Requisição: 1. JMeter envia HTTP (`GET` / `POST`) → 2. Flask abre conexão no PostgreSQL → 3. Banco executa SQL (`SELECT` / `INSERT`) → 4. Flask retorna JSON

4. Metodologia e Cenários de Teste

9 baterias executadas no total (10.800 requisições analisadas)

Carga no JMeter

- **1.200 clientes simultâneos** (threads) em cada bateria.
- **Ramp-up:** 10 segundos.
- 1 requisição por cliente (1.200 requisições por bateria).
- **Métricas avaliadas:**
  - Taxa de Erro (%)
  - Tempo Médio (ms)
  - Percentil 95 / P95 (ms)
  - Vazão / Throughput (req/s)

3 Proporções de Leitura/Escrita

- **Cenário A (50% L / 50% E):** 600 leituras + 600 escritas
- **Cenário B (75% L / 25% E):** 900 leituras + 300 escritas
- **Cenário C (25% L / 75% E):** 300 leituras + 900 escritas

3 Variações do Banco

- **Variação 1:** 100.000 registros **com índices** B-Tree secundários
- **Variação 2:** 20.000 registros **com índices** B-Tree secundários
- **Variação 3:** 100.000 registros **sem índices** secundários (apenas PK)

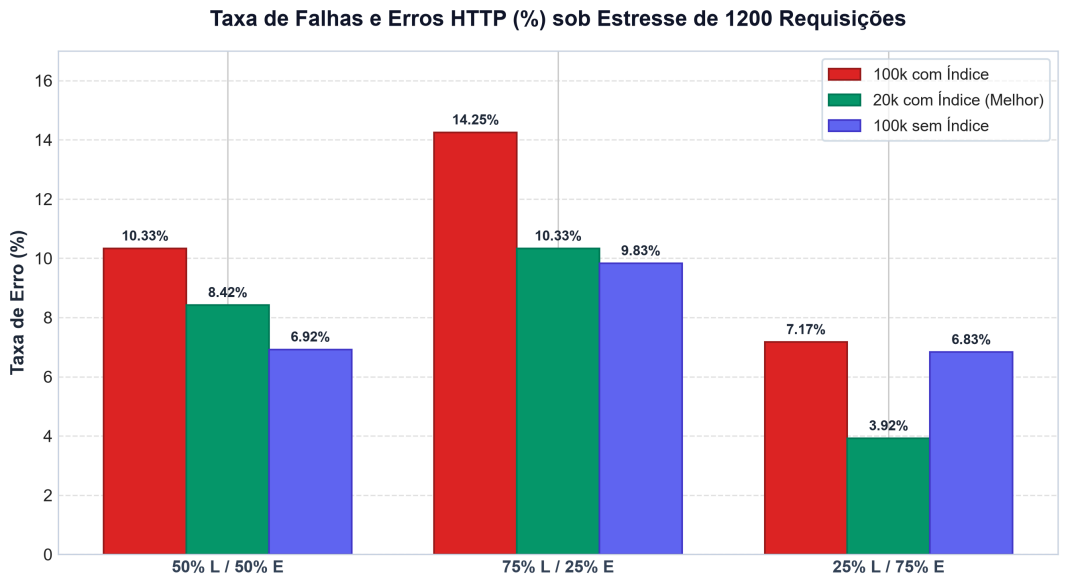
5. Resultados Consolidados (9 Baterias)

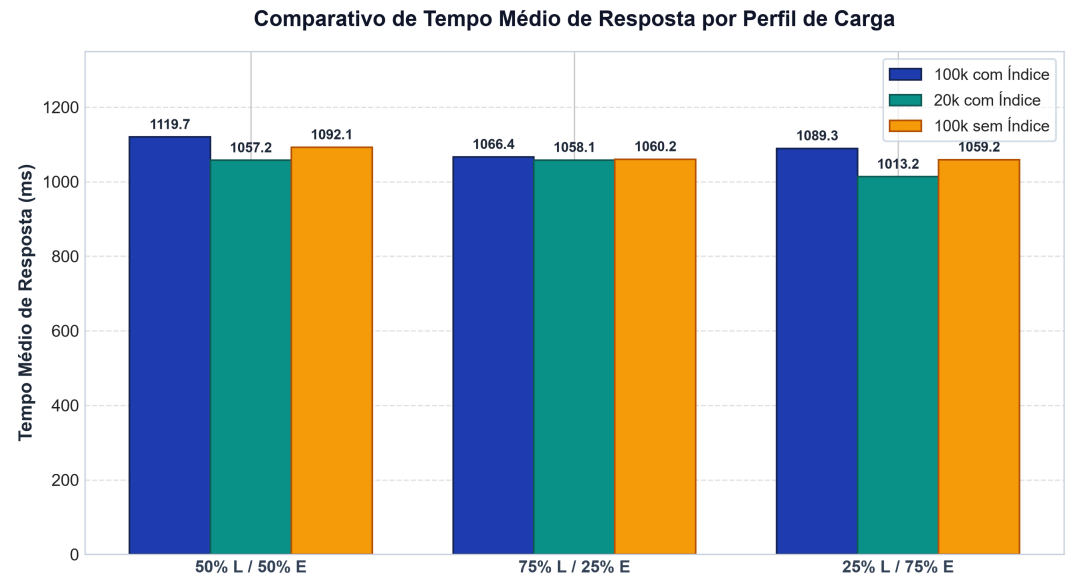
Dados extraídos dos relatórios do Apache JMeter

Configuração do BD	Cenário	Erro Leitura	Erro Escrita	Erro Total	Tempo Médio	Latência P95	Vazão (req/s)
100k com índice	A (50% L / 50% E)	10,00%	10,67%	**10,33%**	1.120 ms	1.613 ms	**106,17 req/s**
100k com índice	B (75% L / 25% E)	14,11%	14,67%	**14,25%**	1.066 ms	1.569 ms	**109,73 req/s**
100k com índice	C (25% L / 75% E)	6,67%	7,33%	**7,17%**	1.089 ms	1.525 ms	**107,85 req/s**
20k com índice	A (50% L / 50% E)	7,33%	9,50%	**8,42%**	1.057 ms	1.609 ms	**105,13 req/s**
20k com índice	B (75% L / 25% E)	10,44%	10,00%	**10,33%**	1.058 ms	1.540 ms	**108,30 req/s**
20k com índice	C (25% L / 75% E)	4,00%	3,89%	**3,92%**	1.013 ms	1.439 ms	**107,58 req/s**
100k sem índice	A (50% L / 50% E)	7,00%	6,83%	**6,92%**	1.092 ms	1.571 ms	**105,03 req/s**
100k sem índice	B (75% L / 25% E)	10,00%	9,33%	**9,83%**	1.060 ms	1.521 ms	**109,00 req/s**
100k sem índice	C (25% L / 75% E)	8,00%	6,44%	**6,83%**	1.059 ms	1.514 ms	**108,03 req/s**

6. Comparativo Gráfico: Taxa de Erros e Latência Média

Comportamento dos 3 perfis de carga nas 3 configurações de banco

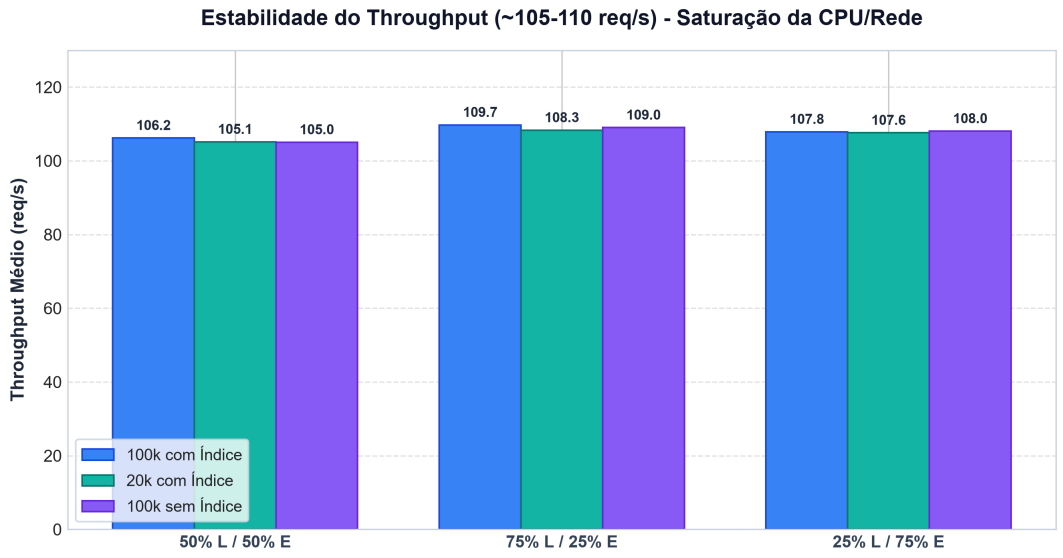




- **Taxa de Erros:** O cenário **B (75% Leitura)** teve mais erros porque cada leitura serializa e retorna 100 registros em JSON, enquanto o cenário **C (75% Escrita)** insere apenas 1 linha por vez.
- **Tempo Médio:** Permaneceu próximo de **1,01 s a 1,12 s** em todas as execuções.

7. Por que a Vazão travou em ~107 req/s em todos os cenários?

Identificação do gargalo principal da arquitetura



A vazão ficou entre 105 e 110 req/s em todas as 9 baterias

O Gargalo está na Camada de Aplicação

- **O que os dados mostram:** Mesmo reduzindo o banco de 100k para 20k ou removendo índices, a vazão não passou de ~110 req/s.
- **Por que isso aconteceu?**
  1. O Flask rodou com `threaded=True` (1 processo criando 1 thread por requisição, com I/O síncrono/bloqueante).
  2. Cada thread executa `psycopg2.connect()` — abrindo uma conexão TCP nova com o banco (sem pool).
  3. Quando mais de 100 threads tentam conectar ao mesmo tempo, batem no limite do PostgreSQL (`max_connections = 100`).
- **Resultado:** Quando as 1.200 requisições chegavam juntas, cerca de 105–110 eram atendidas por segundo e o excedente sofria fila ou erro 500.

8. Pergunta 1: Redução do Dataset para 20k Registros

O que aconteceria com o desempenho se o dataset fosse reduzido de 100k para 20k?

Comparativo nos Testes (100k vs. 20k)

- **Cenário A (50% L / 50% E):**
  - Erro total caiu de **10,33%** → **8,42%**
  - Tempo médio caiu de **1.120 ms** → **1.057 ms**
- **Cenário B (75% L / 25% E):**
  - Erro total caiu de **14,25%** → **10,33%**
  - Tempo médio caiu de **1.066 ms** → **1.058 ms**
- **Cenário C (25% L / 75% E):**
  - Erro total caiu de **7,17%** → **3,92%** (quase metade!)
  - Tempo médio caiu de **1.089 ms** → **1.013 ms**

Explicação Técnica

- **Dados em Memória RAM (Cache):** Com 20.000 registros (base 5x menor), a tabela e os índices cabem com folga no cache de memória do PostgreSQL (`shared_buffers`).
- **Menos Acesso a Disco:** As consultas e inserções exigem menos I/O de disco, terminando mais rápido e reduzindo os erros por timeout.
- **Por que a vazão continuou em ~107 req/s?** Porque a vazão máxima era limitada pela criação de conexões no servidor web Flask, e não pelo tamanho da tabela.

9. Pergunta 2: Adição de Índices no Banco de Dados

O que aconteceu com os índices? Qual índice escolher? O desempenho melhora?

1. Resultado Obtido

- Criamos índices B-Tree em `cidade`, `data_cadastro` e `status`.
- **100k Com vs. Sem Índice:**
  - **A (50/50):** 10,33% vs. 6,92%
  - **B (75/25):** 14,25% vs. 9,83%
  - **C (25/75):** 7,17% vs. 6,83%
- Na prática, a base **com índices teve mais erros** em todos os cenários!

2. Por que isso ocorreu?

- **Leitura sem ganho:** O JMeter enviava `?cidade=Manaus`, mas a API lia `?q=`. A consulta executada não filtrava por cidade. Além disso, a busca por `?q=` usa `ILIKE '%termo%'`, que ignora árvores B-Tree.
- **Escrita mais pesada:** Cada `INSERT` precisava atualizar a tabela + 3 índices B-Tree + log WAL em disco, aumentando a contenção.

3. Qual índice escolher?

- **Para filtro exato de cidade:** Ajustar a API para `WHERE cidade = %s` e usar índice **B-Tree** em `cidade`.
- **Para busca parcial (`ILIKE '%termo%'`):** Usar índice **GIN com trigramas** (`pg_trgm`):

```
CREATE EXTENSION pg_trgm;
CREATE INDEX idx_nome_gin
ON pessoas USING gin
(nome gin_trgm_ops);
```

10. Pergunta 3: Impacto da Escolha do BD (SQL vs. NoSQL)

Como a escolha entre um banco relacional e não-relacional afeta o sistema?

Banco Relacional — SQL (PostgreSQL)

- **Forte Consistência (ACID):** Garante integridade total em cada transação.
- **Esquema Fixo:** Padroniza os campos do cadastro de pessoas.
- **Custo sob Concorrência:** Controle de locks, transações e índices aumenta a latência em picos de escrita.
- **Escalabilidade Vertical:** Escala principalmente aumentando CPU, RAM e disco do servidor.

Banco Não-Relacional — NoSQL (MongoDB / Cassandra)

- **Alta Vazão de Escrita:** Inserções rápidas sem sobrecarga de joins ou constraints relacionais.
- **Escalabilidade Horizontal:** Facilidade nativa para distribuir dados entre vários servidores (*sharding*).
- **Esquema Flexível:** Armazena documentos JSON diretamente.
- **Trade-off:** Geralmente adota *consistência eventual* em vez de ACID rígido.

Conclusão para o nosso sistema: O **PostgreSQL (SQL)** atende muito bem um cadastro estruturado de pessoas na faixa de 100k registros. Um banco **NoSQL** seria vantajoso se a aplicação precisasse absorver milhares de escritas por segundo distribuídas em múltiplos nós.

11. Conclusão e Melhorias Futuras

Principais aprendizados práticos do trabalho

Síntese dos Resultados

- **Vazão (~107 req/s):** O limite de throughput foi definido pela camada de aplicação (conexões síncronas por requisição no Flask), e não pelo PostgreSQL.
- **Dataset Menor (20k):** Reduziu os erros em todos os cenários (chegando a **3,92%** no perfil 25L/75E) por manter os dados em memória RAM.
- **Uso Consciente de Índices:** Índices B-Tree que não casam com o filtro da query não aceleram a leitura e ainda penalizam a escrita.

Como Escalar a Arquitetura?

- **1. Pool de Conexões (Connection Pool):**
  Utilizar `PgBouncer` ou `ThreadedConnectionPool` para reutilizar conexões abertas em vez de conectar a cada requisição.
- **2. Servidor de Produção Assíncrono:**
  Executar a aplicação com **Gunicorn** (múltiplos workers) ou migrar para I/O assíncrono.
- **3. Sintonia entre API e Índices:**
  Aplicar `WHERE cidade = %s` para aproveitar o B-Tree ou criar índice **GIN (** `pg_trgm` **)** para buscas com `ILIKE`.